

Computer Graphics Society

WEBX TASK

Task Submission Deadline : 5TH OCTOBER 2025

Hi, the below tasks are some of the coolest things you can do as a first year! So, even if it feels overwhelming, try to finish whichever you pick up. Pick the task you're most fascinated by and see it through cuz it's all doable!

General Instructions

Tasks Codebase:

- The project codebase shall be well structured
- All the projects shall be able to be locally tested on another device (hence proper GitHub repo setup is a must)
- Appropriate explanation about the task and testing methods should be put in a [README.md](#) file.
- If you are doing multiple projects try to keep them in separate directories (optionally, if you are combining tasks into one then this can be ignored)

Miscellaneous:

- For most of the tasks "Phase 2" is completely optional and is given to introduce you to the higher levels of your projects, adding a level of complexity.
- If you have any kinds of doubts regarding the projects, drop a query in the doubts group. Don't hold yourself back and enjoy the learning experience :)

Submission:

- Wrap up everything in a **PUBLIC** repository on **github** and share the link to that repo.

Resources:

- [Git-Github basics](#)
- Use **gh** for git auth and setup in your local environment for easier commits.

Note: Being comfortable in git is key for development; it allows easy reversion to older versions if issues arise.

- We have tried to include on-demand technologies within the project domains, these might become useful to you at some point in future while doing development work.
- Some might feel overwhelming at first, but keep at it and you'd be surprised to find that most of these are quite easily doable.
- Note: we'll be evaluating **every submission** regardless of it being incomplete. So do remember to submit whatever you come up with over time!

Task Contents:

1. ChatNet: Build a TCP Chat Platform ([Networking](#))
2. Virtual File Compression System ([File handling/compression-decompression](#))
3. Build your first CNN Model ([ML](#))
4. Custom Auth System in [Node.js](#) ([Cryptography/Auth-Systems](#))
5. TermiBot — Your Own Terminal A I Assistant ([APIs and Python](#))
6. *Consensus Mechanism in a Blockchain Network ([Blockchain](#))
7. Digital Locker System ([Security/Storage](#))
8. Personal Whiteboard Web App ([Frontend/Vanilla JS](#))

ChatNet: Build a TCP Chat Platform

Introduction

This challenge takes you through the journey of building your own **chat platform** from scratch using TCP sockets. First, you'll build a simple program where multiple users can chat with each other across ports. Then, you'll level it up by adding **end-to-end encryption** so conversations remain private.

The goal is to get hands-on experience with **socket programming, concurrency, and encryption basics**. No heavy frameworks, just Python and core libraries.

Overview

The task is split into two phases:

Phase 1: Build a simple TCP chat application where multiple clients can send and receive messages over the network.

Phase 2: Add **end-to-end encryption** so messages can only be read by the sender and intended receiver.

By completing both phases, you'll understand how messaging apps work at a fundamental level — from sending raw bytes across a wire to keeping conversations secure.

Objectives

- Learn the basics of **TCP sockets** and how they enable communication.
 - Handle multiple clients using threads or async programming.
 - Build a simple text-based chat app with message routing.
 - (Phase 2) Understand and implement **encryption** to secure chats.
-

Phase 1: Basic TCP Chat

Core Requirements:

- Create a **TCP server** that runs on a chosen port (e.g., 5000).
- Clients should be able to:
 - Connect to the server.
 - Send messages.
 - Receive messages from others in real-time.
- Support **multiple clients** (threading or async I/O).
- Implement simple commands, e.g.:
 - `/quit` → disconnect.

- `/list` → see connected users.
- Messages from one client should be delivered to all others (broadcast) or to specific users (direct).

Tech Stack:

- Language: **Python**
- Libraries: `socket`, `threading` (or `asyncio` if you want to explore async).

Learning Goal:

Get comfortable with **client-server architecture**, handling multiple users, and real-time message exchange.

Phase 2: Secure Chat (Encryption)

Core Requirements:

- Add **end-to-end encryption** so only the sender and intended receiver can read messages.
- Use a beginner-friendly encryption method, e.g.:
 - **Symmetric encryption** with the `cryptography` library (Fernet).
 - Or **asymmetric (RSA)** for key exchange + symmetric for actual messages.
- Ensure that even if someone intercepts traffic, they cannot read the chats.
- Keep the basic chat functionality from Phase 1 intact.

Tech Stack:

- Language: **Python**
- Libraries: `socket`, `threading` / `asyncio`, `cryptography` (Python cryptography package).

Learning Goal:

Understand the basics of **data security in communication systems** and how encryption can protect user privacy.

Optional Extensions

- Add **usernames** and message tagging.
- Add **private chat rooms**.
- Store conversation logs in a text/JSON file.
- Implement a simple **GUI** using Tkinter or PyQt.

- Add **message timestamps**.
 - Explore **asynchronous sockets (asyncio)** for scalability.
-

Resources

Basics:

- [Client-Server Model \(MDN\)](#)
- [TCP vs UDP \(GeeksforGeeks\)](#)
- [What is a Socket? \(IBM\)](#)

Phase 1 (TCP Chat):

- [Python Socket Programming \(RealPython\)](#)
- [Socket Programming in Python \(GeeksforGeeks\)](#)
- [Python threading basics](#)

Phase 2 (Encryption):

- [Python Cryptography Library Docs](#)
- [End-to-End Encryption Basics \(Cloudflare Blog\)](#)
- [RSA Encryption in Python \(Tutorial\)](#)

File Compression System

Introduction

This challenge introduces you to the fundamentals of file compression by building your own

custom compression-decompression pipeline. Instead of using existing algorithms like Huffman coding or ZIP, you'll create lightweight strategies to reduce file size, then restore the original content through decompression.

The focus is on **file handling, string manipulation, redundancy removal, and pipeline design.**

Overview

The task is split into two phases:

- **Phase 1:** Implement a basic text compressor that reduces size by removing extra spaces/line breaks and supports decompression.
- **Phase 2:** Extend the system with custom word-based compression, multiple strategies, CLI options, and compression statistics.

By completing both phases, you'll gain hands-on experience with **file I/O, text processing, and building end-to-end systems.**

Objectives

- Understand and implement file handling in Python.
 - Learn how compression relies on identifying redundancy.
 - Build a pipeline with both compression and decompression.
 - Explore customization, statistics, and error handling in file systems.
-

Phase 1: Basic Compression-Decompression

Core Requirements

- Input: Read a `.txt` file.
- Compression logic:
 - Remove redundant spaces (e.g., multiple spaces → one).
 - Strip unnecessary line breaks.

- Save compressed output to a new file.
- Decompression logic:
 - Restore the file back to its original form (lossless).
- Verify correctness: decompressed file must match the original.

Recommended Tech Stack

- **Language:** Python (required).
- **Libraries:** Only built-in (`os`, `sys`, `argparse`).
- **Approach:** Use string operations (`split()`, `replace()`) and file I/O.

Learning Goal

Get comfortable with **file reading/writing**, **basic compression logic**, and building a **lossless pipeline**.

Phase 2: Advanced Compression Strategies

Core Requirements

- Extend compression with **word substitution**:
 - Replace common/repeated words with short codes (e.g., "`the`" → `#1`).
 - Store mapping dictionary alongside the compressed file.
- Decompression must use the dictionary to rebuild the original.
- Add support for **multiple input files** (batch compression).
- Implement a **simple CLI interface** (file selection + compression mode).
- Show **compression ratio** (original size vs compressed).
- Add **error handling** for corrupted compressed files.

Optional Enhancements

- Visualize stats (e.g., most frequent words replaced).
- Hybrid modes (spaces-only, words-only, or both).
- Support `.csv` and `.log` in addition to `.txt`.
- Handle **large files** efficiently (read/write in chunks).

Recommended Tech Stack

- **Language:** Python

- **CLI:** `argparse` for options (`--file`, `--mode`, `--batch`)
- **Data Storage:** Store mapping dictionary in JSON (`json` module)
- **Stats:** Use `collections.Counter` for word frequencies

Learning Goal

Learn how **dictionary-based compression** works, design flexible strategies, and make the system interactive + user-friendly.

Optional Extensions

- Visual dashboard showing compression stats (matplotlib/streamlit).
- Experiment with different replacement schemes (e.g., character-level codes).
- Compare your custom compressor's ratio vs. `gzip` (just for fun).
- Add password-based encryption to the compressed files (security twist). [Recommended to try if you get time]



ML Track: Build your first CNN

Introduction

This challenge takes you through the journey of building your very first Convolutional Neural Network (CNN) — first for recognizing handwritten digits (0–9), and then for extending it with small improvements to make it more practical and interactive.

The goal is to get hands-on experience with datasets, preprocessing, model building, training, evaluation, and a touch of deployment. The math stays simple — the focus is on learning the ML workflow step by step.

Overview

The task is split into two phases:

Phase 1: Train a CNN to recognize handwritten digits (MNIST).

Phase 2: Add a simple improvement: either data augmentation for robustness, or a tiny interactive demo (Gradio/Streamlit).

By completing both phases, you'll learn how ML projects go from training a model in a notebook to actually using it.

Objectives

- Understand how to load and preprocess image data.
 - Build and train a CNN from scratch.
 - Evaluate the model using accuracy and a confusion matrix.
 - Save and reload trained models.
 - (Phase 2) Add either robustness (via augmentation) or usability (via a demo).
-

Phase 1: Handwritten Digit Classifier

Core Requirements:

- Use the **MNIST dataset** (28×28 grayscale digits, 0–9).
- Preprocess: normalize images to [0,1].
- Build a **CNN** in TensorFlow/Keras or PyTorch.
- Train with categorical cross-entropy loss, track accuracy.
- Save the **best model checkpoint**.
- Evaluate on a test set, and show:
 - Test accuracy.
 - A **confusion matrix** of predictions.
- Write a small inference script (CLI or notebook cell) that:
 - Takes an image file.
 - Outputs the predicted digit + confidence score.

Learning Goal:

Get comfortable with the full ML loop: dataset → preprocessing → CNN → training → evaluation → inference.

Phase 2: Make It Better (completely optional):

- **Augmentation:** Add small transformations (rotation, noise, shifts) to improve robustness.
- **UI Demo:** Build a tiny interactive test using **Gradio** or **Streamlit**, where you can draw/upload digits and see predictions.
- **Visualization:** Show feature maps or misclassified examples to understand the model's behavior.

Learning Goal:

Learn how to extend a basic ML project, either by making it more robust or by making it more usable.

Evaluation Criteria:

- **Phase 1 Correctness** — Model trains, inference works, test accuracy reasonable for MNIST (baseline expectations).
- **ML Practices** — Proper preprocessing, train/val/test split, checkpointing, reproducibility (seed/config).
- **Evaluation & Analysis** — Confusion matrix, error cases, explained tradeoffs.
- **Code Quality & Documentation** — Clear README, runnable scripts, dependency list.
- **Phase 2 / Bonus** — Alpha-numeric extension, UI demo, augmentation, visualizations.

Optional Extensions

- Deploy your model on Hugging Face Spaces or Streamlit Cloud.
 - Add a drawing pad for live digit predictions.
 - Convert your model to **ONNX/TFLite** for lightweight usage.
 - Visualize CNN feature maps for interpretability.
 - Train on a harder dataset (Fashion-MNIST).
-

Resources

Basics:

- [What is a CNN? \(GeeksforGeeks\)](#)
- [MNIST Dataset on TensorFlow/Keras](#)

Phase 1:

- [Beginner CNN with Keras \(Medium\)](#)
- [PyTorch MNIST Tutorial](#)

Phase 2:

- [Data Augmentation in Keras](#)
- [Gradio Quickstart \(spin up a quick interface for ML api\)](#)
- [Streamlit Docs](#)

Build a Custom Auth System in Node.js (No External Auth Libraries)

Introduction

Authentication is everywhere — but most developers just install `jsonwebtoken` and move on. Here, you'll build your own JWT-inspired auth system from scratch, with a twist: we'll use a custom token format, enforce role-based access, add time-based restrictions, and provide an introspection endpoint.

This challenge teaches you how real-world auth systems work under the hood and how design choices affect security.

Overview

You'll implement a token-based authentication system using only Node.js built-ins (e.g., `crypto`, `http`, `buffer`).

- **Phase 1:** Build the core token system and secure endpoints.
 - **Phase 2:** Add introspection, revocation, and extra access controls.
-

Objectives

- Learn cryptographic signing and verification with Node's `crypto`.
 - Understand token serialization, encoding, and validation.
 - Implement real-world concepts: expiry, role-based access, time-based restrictions.
 - Practice clean code design, modularization, and security hygiene.
-

Core Requirements

Custom Token Format

Token = `<base64url({header, payload})>.<signature>`

- **header** contains: `{ "alg": "HS256", "typ": "CJWT" }` (CJWT = "Custom JWT")
- **payload** contains: `{ "iss": "...", "sub": "...", "role": "...", "iat": ..., "exp": ... }`
- **Signature** = HMAC-SHA256 over the base64url-encoded `{header, payload}`.

Role-Based Claims

- Token must include a `"role"` field (e.g., `"user"`, `"admin"`).
- Certain endpoints require `"admin"` role.

Time-Based Access Control

- Token payload must support a `"validHours": [startHour, endHour]` claim.
- Example: `{ "validHours": [9, 17] }` → Token only works during 9 AM – 5 PM server time.

Introspection Route (Phase 2)

- Provide `/introspect` endpoint.
- Accepts a token and returns whether it's valid + decoded claims.

Phases

Phase 1 — Core Token System

Goal: Build a working authentication system with the custom token format.

Requirements:

- Implement helpers:
 - `sign(payload)` → returns `<base64url({header,payload})>.<signature>`
 - `verify(token)` → checks signature, expiry, role, and validHours.
 - `decode(token)` → returns claims without verification.
- Build an HTTP demo:
 - `POST /login` issues token with role + expiry.
 - `GET /protected` requires a valid token.
 - `GET /admin` requires `"role": "admin"`.
- Ensure:
 - Expiry (`exp`) is enforced.
 - Signature verification uses `crypto.timingSafeEqual`.
 - Tampered tokens are rejected.

Learning Goals:

- Implement HMAC-SHA256 signing from scratch.
- Practice base64url encoding/decoding.
- Understand role-based and time-based access control.

Phase 2 — Robust Features

Goal: Extend the system with introspection and session management.

Requirements:

- Add `/introspect` route (basically, to check is the token valid? And what does it allow?):
 - Input: token.
 - Output: `{ "valid": true/false, "claims": {...} }`.
- Add token revocation:
 - Maintain an in-memory blacklist of revoked tokens.
 - Add `POST /logout` to revoke the current token.
- Add basic key rotation:
 - Support multiple signing keys with `"kid"` in header.
 - New tokens use the active key; old ones validate until expiry.

Optional Extensions (pick if you have time):

- Implement refresh tokens with rotation.
- Store reason codes for revocation (e.g., `"user logout"`, `"admin revoke"`).
- Add structured logs for login/verify failures.
- Add additional claims like `aud` or `jti`.

Learning Goals:

- Build introspection tools to debug tokens.
- Implement revocation and session invalidation.
- Practice designing extensible, secure systems.

Deliverables

- **token.js**: implements sign/verify/decode.
- **server.js**: provides `/login`, `/protected`, `/admin`, `/introspect`, `/logout`.
- **README.md**: documents design decisions, routes, and test cases.
- **Basic test cases** covering tampered tokens, expired tokens, invalid hours, and role enforcement.

Evaluation Criteria

- **Correctedness**: auth system should work as intended with proper fallback and error messages in case of invalid auth.
- **Constraint**: no auth libraries should be used for making the *auth system*
- **Reasoning**: understanding of your code is a must, you must be able to customize your

project as per requirements.

Resources

Foundational Concepts

- [Node.js crypto module \(Official Docs\)](#)
- [What is base64url encoding? \(Auth0 Blog\)](#)
- [Introduction to HMAC \(RFC 2104\)](#)

Phase 1 (Core Token System)

- [Node.js Buffer class](#)
- [HTTP module in Node.js](#)
- [Timing attack safe comparisons \(Node crypto docs\)](#)
- [JWT Explained \(without libraries\) – Blog](#)

Phase 2 (Robust Features)

- [Key Rotation Best Practices \(Auth0\)](#)
- [Token Revocation \(OAuth 2.0 RFC 7009\)](#)
- [Blacklisting vs Whitelisting Tokens – Blog](#)
- [Introspection Endpoint \(RFC 7662\)](#)



TermiBot — Your Own Terminal AI Assistant

Introduction

Anyone can chat with an LLM through a website. But a real developer should know how to **build**

their own assistant, right in the terminal.

In this challenge, you'll create **TermiBot**: a Python-based, interactive chatbot that talks to an LLM API. But don't stop at a barebones script — you'll make it **robust, useful, and customizable**, with proper error handling, logging, and features that make it feel like a real tool.

The twist? You'll be working under real-world constraints: API budgets, rate limits, and design trade-offs.

Overview

The task is divided into **two phases**:

- **Phase 1: Core API + REPL** — Get a basic chatbot working in the terminal using an LLM API. (wrapper around any existing llm)
 - **Phase 2: Robust Terminal Assistant** — Extend it into a practical, developer-friendly tool with features, logging, and error handling.
-

Objectives

By the end, you'll:

- Learn how to **call and handle external APIs** from Python.
 - Practice **secure key handling**, JSON parsing, and error recovery.
 - Build a **command-line interface (CLI)** with interactive features.
 - Apply **Python design principles** (modularity, reusability, clarity).
 - Create a **useful personal tool** that you could keep using daily.
-

Phase 1: Core API + REPL

Requirements

- Load API key securely (e.g., environment variables).
- Create a REPL loop:
 - User enters a prompt.
 - Program sends request to an LLM API.

- Response is displayed in the terminal.
- Handle at least these errors:
 - Invalid API key.
 - Network errors (simulate with retries).

Deliverables

- A script (`termibot.py`) that runs and lets you chat.
- Clear documentation on how to set up and run.

Learning Goal: Get comfortable with making API calls, parsing responses, and building a basic interactive CLI in Python.

Phase 2: Robust Terminal Assistant

Requirements (any number of these from below):

- **Enable multiple LLM's:**
 - Use multiple LLM api's and enable the user to switch between them using flags
 - e.g. `termibot.py --gemini` or `termibot.py --deepseek`
- **CLI Enhancements:**
 - Colored output (different colors for user and bot).
 - At least **two commands**:
 - `/save filename.txt` → save chat history.
 - `/help` → list available commands.
- **Logging:** Save chat history automatically in a structured format (timestamp, role, message).
- **Error Handling:**
 - Handle rate limits gracefully (simulate if needed).
 - Implement retry with backoff on failed requests.
- **Code Structure:** Split into modules (e.g., `api_handler.py`, `cli.py`, `utils.py`).
- **Custom Feature:** Add at least one feature of your choice, e.g.:
 - `/summarize file.txt` → summarize a local file.
 - `/explain code.py` → explain a code snippet.
 - `/mode code` → switch chatbot personality/mode.

Deliverables

- A modular Python project with clear directory structure.
- Documentation on features, commands, and usage examples.

Learning Goal: Write maintainable Python code, build error-resilient systems, and add creative utility features.

Evaluation Criteria

- **API Fluency** : Proper API use, secure key handling, error handling.
- **Python Code Design** : Modular, clean, documented.
- **CLI Usability** : Clear interface, colors, commands, smooth interaction.
- **Robustness** : Handles failures, logs properly, survives bad inputs.
- **Creativity / Custom Feature** : At least one original feature beyond the basics.

Example Interaction with the terminal chatbot:

Sample Interaction

```
>>> Welcome to TermiBot! Type your message or /help for commands.
```

```
You: Hello TermiBot!
```

```
Bot: Hi there! I'm ready to assist you right from your terminal.
```

```
You: Can you explain what decorators are in Python?
```

```
Bot: Sure! A decorator is a function that modifies the behavior of another function...
```

```
You: /save notes.txt
```

```
[System] Chat saved to notes.txt
```

```
You: /summarize file.txt
```

```
Bot: The file contains an overview of TCP networking basics...
```

Resources:

Fundamentals:

- [What is an API by freecodecamp](#)
- [A gentle introduction to JSON](#)
- [Python venv Doc](#)

Phase 1:

- You can use any LLM api for the said purpose, if not sure then follow the Google official docs for gemini API or any other well documented LLM APIs
- [Handling secret keys and environment variables](#)

Phase 2:

- [Rich library for color terminal UI](#) (you may use any other library for colored terminal outputs)

Consensus Mechanism in a Blockchain Network

Introduction

A blockchain is fundamentally a linear data structure in which data is appended following stringent procedures that require either substantial computational resources or a process of collective agreement and trust. This can be likened to the passage of a bill through various legislative bodies, where approval is not guaranteed at each stage. Only after successfully passing all levels of scrutiny is the bill incorporated into the constitution.

In the context of a blockchain, data is organized into discrete units referred to as “blocks.” Each block contains a reference (pointer) to the subsequent block, collectively forming what is known as a “chain.”

Not every proposed block is accepted. Instead, new blocks undergo an approval process before becoming a permanent part of the chain.

Approval Process for Blocks

In certain blockchains, such as Bitcoin, individuals known as miners are responsible for proposing new blocks. These miners must solve complex mathematical puzzles, and the first to do so earns the right to add their block to the blockchain (represented by their wallet address, as blockchain identities remain anonymous). In addition, successful miners receive an incentive for their contribution.

To draw an analogy, consider a king seeking a new advisor. Numerous candidates apply, but only the one who is able to answer all the king’s challenging questions is selected. Similarly, only blocks that meet stringent requirements are approved and added to a blockchain.

The method through which blocks are approved and added to the blockchain is called a “consensus mechanism.” There are multiple types of consensus mechanisms, further details of which can be found in the provided resources.

Task Overview

For this task, participants are required to propose a new consensus mechanism or implement an existing one (except Proof of Work) in the BlockRS blockchain simulation environment.

Participants are permitted to utilize Large Language Models (LLMs) and consult reputable online resources to support their work on this task.

About BlockRS

BlockRS is a blockchain simulation platform that facilitates the creation and management of blockchain networks locally on your computer.

Repository and Documentation: Reference the repository at <https://github.com/shrehanrajsingh/blockrs.git>. Please review the README file thoroughly for

complete installation instructions. Additional guidance is available in the `docs/video` directory, where a step-by-step video tutorial can be found.

Environment Setup

Ensure that Docker and Git are installed on your system before proceeding.

1. Clone the Repository:

```
git clone --recurse-submodules https://github.com/shrehanrajsingh/blockrs.git
```

2. Navigate to the Project Directory:

```
cd blockrs
```

3. Build the Docker Image:

Ensure Docker is operational, then run:

```
docker build -t blockrs .
```

4. Further Instructions:

After building the Docker image, follow the README and check the `docs/video` directory to interact with and test the project.

Implementation Guidance

- Review the `blockchain/consensus` directory.
- Examine the files `proofofwork.hpp` and `proofofwork.cpp` to understand how the Proof of Work mechanism (used in Bitcoin) is structured.
- Add your own implementation of a consensus mechanism within this directory. You may choose to implement any existing consensus protocol, except Proof of Work.
- For reference, consult `consensus.hpp`, which provides the class structure required for new consensus mechanism implementations.

For example, consider the declaration of ProofOfWork class:

```
namespace rs::block
{
    using namespace json;

    /**
     * For this implementation of ProofOfWork
     * we will use Bitcoin's PoW algorithm
     * which involves taking the hash of the
```

```

    * previous block and concatenating a
    * number `nonce` to it.
    * If the hash of the resultant string
    * starts with `difficulty_target` number of zeroes
    * then the nonce is valid
    */
class ProofOfWork : public Consensus
{
public:
    ProofOfWork ();
    ~ProofOfWork ();

    json_t operate (json_t &) override;
    json_t compute (json_t &) override;
    json_t next (json_t &) override;
};
} // namespace rs::block

```

The ProofOfWork class, as demonstrated in `proofofwork.hpp`, inherits from the base `Consensus` class defined in `consensus.hpp`. It contains three methods, with only the `compute` function requiring implementation.

Now let us take a look at it's definition (simplified)

```

json_t
ProofOfWork::compute (json_t &data)
{
    if (!data.has_key ("prev_hash") || !data.has_key ("difficulty_target"))
        throw std::invalid_argument (
            "Please provide 'prev_hash' and 'difficulty_target'");

    int nonce = 0;
    std::string ph = data["prev_hash"]->as_string ();
    int dt = data["difficulty_target"]->as_integer ();
    std::string pref (dt, '0');

    std::string cur;
    uint8_t h[32];
    cur = ph + std::to_string (nonce);

    SHA256 (reinterpret_cast<const uint8_t *> (cur.c_str ()), cur.size (),
h);

    size_t num_0;
    while (1)
    {
        /**
         * For performance we will omit

```

```

    * the use of operate and next
    * since both require working with
    * json which is resource intensive.
    * For algorithms involving the
    * use of a lot of variables,
    * it becomes better to define and
    * use operate and next
    */
/* next */
num_0 = 0;
while (num_0 < 32 && h[num_0] == '0')
    num_0++;

dbg ("num_0: " << num_0 << "\tdt: " << dt
    << "\nhash: " << to_hex (h, 32));

/* operate */
if (num_0 == dt)
    break;

nonce++;
cur = ph + std::to_string (nonce);

SHA256 (reinterpret_cast<const uint8_t *> (cur.c_str ()), cur.size
()),
    h);
}

json_t res;
J (res["nonce"]) = nonce;
J (res["hash"]) = to_hex (h, 32);

return res;
}

```

Explanation of the routine

The `compute` method operates using cryptographic hashing. For Bitcoin, the process is as follows:

1. Retrieve the hash of the entire blockchain.
2. Append a unique number (called a “nonce”) to this hash.
3. Compute the hash of the resulting string.
4. Ensure the resulting hash begins with a specified number of zeros (this is the “difficulty target”; BlockRS uses a target of two zeros for expediency, although Bitcoin uses up to five).
5. If the condition is met, the block is accepted.

6. If not, increment the nonce and repeat the process.

Consensus mechanisms may utilize a range of criteria for block approval, from relatively simple rules (e.g., accepting hashes containing three 'a' characters) to more complex processes (e.g., voting among network participants).

The aim of this task is to provide practical experience with complex workflows and real-world implementations of blockchain technologies.

Please ensure your consensus mechanism is innovative yet clear, and rigorously tested within the BlockRS framework.

Resources

- [BlockRS Blockchain Simulator](#)
- [Mastering Blockchain, 2nd Edition](#)
- [Bitcoin Whitepaper](#)

Support and Contact

For further assistance, please contact:

- **Shrehan Raj Singh**
Email: shrehanofficial@gmail.com
Mobile: +91 9475005020



System Track: Digital Locker System

Introduction

This challenge is about designing a **digital locker application** where multiple users can securely store and retrieve personal items like notes, reminders, or credentials. The lockers are protected by a **PIN**, simulating basic authentication and secure storage.

The focus is on **user authentication, persistent storage, and safe item retrieval**.

Overview

The task is split into two phases:

- **Phase 1:** Build a simple single-locker system where users can register, log in with a PIN, and store/retrieve items.
- **Phase 2:** Extend the system with multiple users, categories, better security, and optional features like encryption or expiry.

By completing both phases, you'll learn the **basics of authentication, file handling, and simple security design**.

Objectives

- Implement PIN-based authentication.
 - Store and retrieve user items persistently (not lost after exit).
 - Prevent unauthorized access.
 - Extend functionality with categories, multiple users, and basic encryption.
-

Phase 1: Basic Locker System

Core Requirements

- Create a simple locker for one user.
- User registers with a PIN.
- User can:
 - Add an item (as text).
 - View all stored items.

- Remove an item.
- Items must persist between program runs (store in JSON or text file).
- Every access requires correct PIN entry.

Learning Goal

Get comfortable with **file handling (JSON/text)**, **basic authentication**, and **simple CRUD operations** for stored items.

Phase 2: Multi-User & Secure Locker

Core Requirements

- Support multiple users, each with a **unique locker ID + PIN**.
- Prevent unauthorized access to other users' lockers.
- Add categories for items (e.g., Passwords, Notes, Docs).
- Implement PIN verification for every sensitive action.
- Add optional features:
 - Simple encryption for stored items (Caesar Cipher, XOR, AES).
 - Item expiry (auto-delete after a set time).
 - Basic GUI (Tkinter, PyQt or web-based).

Learning Goal

Understand **multi-user authentication**, improve **security awareness**, and practice **system design** for small applications.

Evaluation Criteria

- **Functionality** – Locker works, items stored/retrieved correctly.
 - **Security** – PIN verification, isolation between users.
 - **System Design** – Clean, modular, scalable code.
 - **Code Quality** – Readable, commented, documented.
 - **User Experience** – Easy-to-use menu/CLI (or GUI if added).
-

Optional Extensions

- Encrypt PINs before storage.
- Add a search feature for items.

- Visualize storage usage (how many items, categories).
 - Export/import locker data as backup.
-

Recommended Tech Stack (for beginners)

- **Language:** Python (required).
- **Storage:** JSON (`json` module, easy key-value store).
- **Authentication:** Basic PIN check (store hashed PIN with `hashlib` if you want extra credit).
- **CLI:** Use `input()` and a menu-driven loop.
- **Optional GUI:** Tkinter (beginner-friendly).
- **Encryption:** Simple Caesar Cipher or XOR (manual implementation).



Personal Whiteboard Web App

Introduction

This challenge is about designing a **personal whiteboard application** where a single user can draw, erase, and manage sketches directly in the browser. The whiteboard should persist its state so that drawings are not lost after a page refresh or reopening.

The focus is on using the **Canvas API**, implementing **basic drawing tools**, and ensuring **local persistence** with storage mechanisms like `localStorage`. The task emphasizes core frontend skills such as event handling, UI design, and state management.

Overview

The task is divided into two phases for progressive development:

Phase 1: Build a basic single-user whiteboard with essential drawing tools and persistence.

Phase 2: Extend the system with advanced features such as undo/redo, shapes, dark mode, and export options.

By completing both phases, you will gain practical experience in **graphics programming with the Canvas API**, **frontend event handling**, and **data persistence techniques**.

Objectives

- Implement a canvas where users can draw freehand.
 - Add basic tools: pencil, eraser, clear board.
 - Persist whiteboard data across sessions.
 - Extend functionality with multiple tools, themes, and export features.
-

Phase 1: Basic Whiteboard

- Create a drawing area using the HTML Canvas API (don't use other libraries). Canvas size should at least be 600px X 400px.
- Implement basic tools:
 - Pencil (freehand drawing)
 - Eraser
 - Clear board
- Allow the user to select stroke color and thickness.
- Persist the whiteboard state locally using **localStorage**, ensuring the drawing is restored after refresh or reopening.

Phase 2: Enhanced Whiteboard

- Add **undo/redo** functionality for user actions.
- Support simple shapes insertion (rectangles, circles, lines).
- Implement a **dark/light mode toggle** for improved UI/UX.
- Enable **export** of the whiteboard as an image (PNG). (**Bonus: Upscale the image before saving to ensure better quality**)
- Ensure the application is **responsive**, working smoothly on both desktop and mobile. The canvas should adjust to screen size changes while maintaining drawing scale. Consider using `window.resize` events and scaling existing drawings accordingly.

Evaluation Criteria

- **Functionality** – Core drawing, erasing, and persistence features work correctly.
- **UI/UX** – Clean interface and intuitive toolbar.
- **System Design** – Code is modular, structured, and scalable.
- **Code Quality** – Readable, sufficiently commented, consistent formatting, and meaningful naming. Don't over-comment code.
- **Polish & Extensions** – Improvements that enhance core features, such as smoother drawing, better eraser precision, customizable stroke sizes and colors, undo/redo functionality, and more intuitive toolbar and UI interactions.

Optional Extensions

- Multiple boards (saved as separate files).
- Keyboard shortcuts for tools.
- Allow drawings to be separated into layers, similar to design tools like canva.

Tech Stack

- **Frontend:** Native HTML5, CSS3, JS. You can ES6+ features.
 - **Canvas:** Native Canvas API.
 - **Storage:** localStorage.
 - **Styling:** Plain CSS.
 - **Deployment:** Netlify / Vercel / GitHub Pages (**Bonus**)
-

Submission Format

- 📁 **GitHub Repository** (public) containing source code.
 - 📄 **README.md** with:
 - Setup instructions
 - Features implemented
 - Screenshots/demos (if possible)
 - 🌐 **Deployed Link (Bonus)**
-

Resources

Canvas & Drawing

- MDN Web Docs – Canvas API [Canvas API](#)
- HTML5 Canvas Tutorial (w3schools): [HTML Canvas Graphics](#)

Storage & Persistence

- MDN – localStorage: [Window: localStorage property](#)

Advanced Features

- Undo/Redo in Canvas: <https://stackoverflow.com/questions/11253156/how-to-implement-undo-redo-functionality-in-html5-canvas>
- Export Canvas to Image: [HTMLCanvasElement: toDataURL\(\) method](#)
- Dark Mode in CSS: [prefers-color-scheme - CSS | MDN](#)

Deployment

- Netlify Deployment Guide: [Overview | Netlify Docs](#)
- Vercel Deployment Guide: [Deploying to Vercel](#)
- GitHub Pages Guide: [Quickstart for GitHub Pages](#)

Example Code: Exporting a Canvas as PNG

The following function demonstrates how to export the content of an HTML canvas as a PNG file that the user can download:



```
1 function exportCanvasAsPNG(canvas, fileName = "whiteboard.png") {  
2     const link = document.createElement("a");  
3     link.download = fileName;  
4     link.href = canvas.toDataURL("image/png");  
5     link.click();  
6 }
```